

Konrad Szulc, Lawin Khalil,
Stefan Beatović, Zeljka Zujic

Lab Work Report
Introduction to Mobile App Design and Development
R0334-3014 / Kalpana Rathod

Laurea University of Applied Sciences
27.10.2025

Lab #4: Demo Mobile App with Ionic Framework and Firebase Authentication

Laurea University of Applied Sciences
27.10.2025

Table of Contents

1	GitHub Repository	3
2	Learning Outcome and Reflection	3
	2.1 Konrad.....	3
	2.2 Lawin	4
	2.3 Stefan	5
	2.4 Zeljka.....	5
3	App idea and collaboration methods	6
4	Starting the project	6
5	Using Firebase Authentication	8
6	Working with Firestore	12
7	Styling and UX	17
	References	25

Laurea University of Applied Sciences
27.10.2025

1 GitHub Repository

As a team we all collectively worked on one GitHub Repo during the formation of the app, which the link is: <https://github.com/konradszulc/Labwork-4MADD>

We are Team 4 consisting of Lawin Khalil, Konrad Szulc, Zeljka Zujic, and Stefan Beatovic.

2 Learning Outcome and Reflection

2.1 Konrad

As this course continues, so does the complexities of learning to interact with various components, features and other software related to creating an app in Ionic. In this module, it was my first time hearing and learning with Firebase, but it intrigued me quite a bit, as it is very useful for authenticating and storing user's data. Firebase provides numerous docs to the many practical uses of their software, thus, making it easier for a beginner such as myself to learn. I also utilized the material provided in this module to understand the process of creating and using firebase, to integrate it into this app. For example, the video of creating a registration page by Mehul - Codename (2020) provided a great foundation for a page to be accessed by firebase authentication, showing demonstration to how it works. As well, the other following videos in the series provided were ideal in putting the concepts to practice. However, the videos were also quite outdated, so it took trial and error, alongside the docs provided to understand the new ways to use Firebase within Ionic.

This proved useful for it helped me to get more familiar with Firebase authentication and create functioning pages to register/login. By understanding these basic concepts, the skills can be transferrable to using the other features of Firebase, such as Firestore database, like creating functions within a separate file for Firebase, or utilizing different Firebase projects to where to get the right API Key or IDs. As well managing the settings on the Firebase project itself to allow users access or what type of authentication required for the app. Providing insight to how versatile this program can be for many different ideas.

As a team, we met early on within the project timeline to discuss what type of app we would like to create that fulfils the criteria for this assignment. As well, assigning each other tasks and practice trying to make the required pages. However, we also recognized the importance of collaboration when learning new codes and concepts, thus we all put effort into each page, and had all files in a single GitHub repository to push and pull from, noticing each other's changes with commit messaging. Though I have had already now some experience with Git and Github, learning to use it with others was an eye-opening experience as there can be quite troubles with merging or losing code if not handled properly. I still believe there is much for me to learn, as potentially creating test branches before pushing to the main branch. Nonetheless, working in a group was especially beneficial to learn from one another in help to get the programming running smoothly. Generally, I would run

Laurea University of Applied Sciences
27.10.2025

into some various problems such as trying to get a button a certain location, or when redirecting to a page when a certain action is accomplished. With the help of my teammates, these problems were rectified. I would also say that my understanding of typescript and react hooks, is getting better, I grasp the understanding of useEffect and declaring the data type of a variable before called upon, to have reactive and functional apps. Overall, it was a great learning experience, good feeling of overcoming obstacles in the code, and helpful collaboration with my teammates bringing excitement for the next projects we work on together.

2.2 Lawin

This course has been a true testament of perseverance considering the challenges faced in everyday life and dealing with the complexities of this module. It has raised my anxiety levels in many ways, but I have been fortunate to have such incredible team members who have supported me in bridging the knowledge gap to perform better in our collective project.

I started this course not having any knowledge about Ionic React Framework and Firebase but through countless trial and error attempts, I can say I have built some level of understanding and expertise about the life cycle of mobile application development. I progressed creating a login page but faced some hurdles over which my teammates provided insightful inputs to help me overcome the barrier. In addition, I understand using TypeScript better which was one of my main goals for this course. I also improved my skills using GitHub as a team collaborating on the project. Not only that, but I have also learnt better in how UX design can be applied, for example, considering Ow Zone and handedness in mobile app development. The images below show, for example, how placing the buttons in the “natural” zone with large padding helps with handedness.

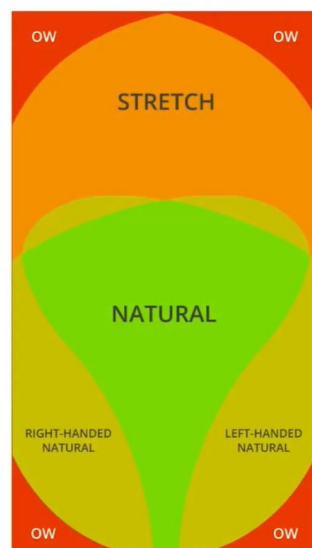


Figure: Ow Zone (Udacity)

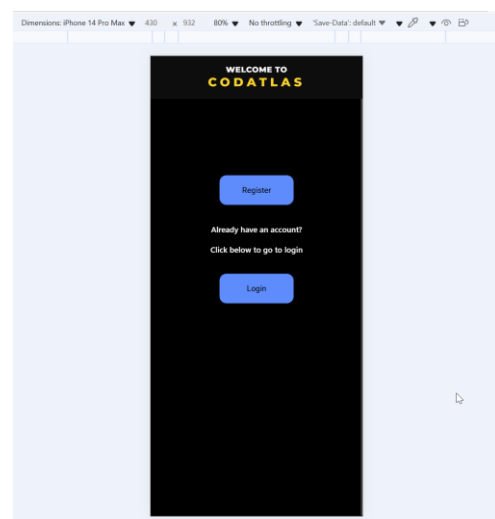


Figure: Our application Codatlas home screen

Laurea University of Applied Sciences
27.10.2025

2.3 Stefan

In this module, I was introduced to Firebase for the first time, and it quickly became one of the most interesting tools I've used so far. Learning how to integrate it with an Ionic app for user authentication and data storage was both challenging and rewarding. Although some of the tutorials were outdated, I relied on Firebase's documentation and trial and error to build working registration and login pages. This experience also helped me understand how to manage API keys, authentication settings, and other Firebase features.

Working as part of a team enhanced the learning process even more. We planned our app together, divided tasks, and collaborated through a shared GitHub repository. This improved my understanding of version control and teamwork, especially when dealing with merging issues and code updates. While doing this project, we communicated actively through Teams and WhatsApp.

Overall, this module strengthened both my technical and collaborative skills. I've gained confidence in using TypeScript, React Hooks, and Firebase, and felt a genuine sense of accomplishment when our app came together through shared effort and problem-solving.

2.4 Zeljka

As the practical assignments for this course keep getting progressively more challenging, we are encouraged to build upon our knowledge and improve our skills. This lab helped me to better understand certain concepts in developing Ionic React applications. For example, I am more confident in how to use `useState` and `useEffect` hooks from React and more familiar with TypeScript syntax. Additionally, I have a clearer picture of how Ionic mobile application development actually works, more specifically, how different frameworks are integrated to result in a native-like application.

What was new and specific to this lab work was Firebase. It was very interesting to see how much such a platform makes application development easier; without additional knowledge of back-end development, we can establish authentication, create a database, and store user data. Interacting with the Firebase console was pretty straightforward. However, understanding how Firestore and its components (collections and documents) can be used inside our project required some work and additional reading.

Another valuable takeaway from this assignment was working with Git. I already had some experience using Git in the group project from last semester's Introduction to Programming course. However, this experience with using Git was much smoother, mostly due to the practice we already had in our previous labs and the skill of my teammates.

Apart from the provided materials, I found some additional resources that were valuable for creating certain components. For example, the tutorial from Didin (2025) was helpful when setting up Firestore and integrating the database, while the video on IonFab explained how to use this UI component correctly and make the landing page more user-friendly (Tech Assembler 2020).

Finally, the overall teamwork experience was positive and beneficial. As a group, we first met in Teams to discuss the ideas for our app and the course of action. Additionally, we created a WhatsApp group to make communication more efficient. Then, our teammate created the repository that we would use in our project, so we all got to practice staging and committing changes, as well as synchronizing local and remote repositories. After we were confident in how we wanted to collaborate, we proceeded with our tasks. The perk of working in group settings is having multiple perspectives on a problem or idea. Collaborating with teammates who are willing to share knowledge, compromise and distribute responsibilities made this assignment much more interesting than if it were an individual activity.

3 App idea and collaboration methods

After the first group meeting, we decided to create a simple app named Codeatlas for keeping a collection of code snippets. Our initial idea was to design several pages to demonstrate the app's core functionalities (login, registration and saving data to a database), in other words, to create a minimum viable product that we can continue developing in future labs). After agreeing on the app idea, we decided on our collaboration methods, which included communication over Teams and WhatsApp, creating and cloning a GitHub repository where we stored our Ionic React project and practicing staging, committing, and pushing changes.

4 Starting the project

The next step was installing Firebase and creating a new project using the Firebase Console, followed by configuring Firebase in Visual Studio Code. We initialized the Firebase app using the configuration settings provided in our project settings in Firebase.

Then we created the pages for our app. Apart from the Home page, which provides users with an option to choose between login or registration (depending on whether they already have an account), we created a Login page, a Registration page, a Landing page (the main page of our app where saved snippets are displayed), and an Add Snippet page that displays UI for adding user input and saving snippets. Inside the pages, we used the `ir-page-component` shortcut from Ionic Snippets Extension to generate the code outline that we used as a starting point.

In the future, we plan to add an About page and a separate Edit Snippet page. However, we decided that the current pages were the most important for

demonstrating the core functionality of our app. Additionally, we wanted to have simple navigation within the pages, that can easily be added to in the future. We decided to have the menu navigation within the App.tsx to have it consistent throughout the whole app. This way we can easily show the navigation menu on any pages we create in the future.

The menu navigation also includes some interesting connections with Auth, such as displaying the email of the current logged in user, which we were able to do through a useEffect, which changes based on when the Auth changes (onAuthStateChanged).

We also handled routing to create a smooth navigation between pages. However, this section still needs some improvement, as we plan to create a conditional routing for certain cases. For instance, when a user who is not logged in attempts to access the Landing page, the current setup allows this activity. We intend to solve this issue in one of our future labs.

Instal Firebase Packages

```
\Labwork-4\mobileTeam4>npm install firebase
```

Laurea University of Applied Sciences
27.10.2025

Menu Functionality in App.tsx

```
App.tsx X
src > App.tsx > App
49 const App: React.FC = () => {
50   async function signedOut() {
51     if (resolution) {
52       setIsOpen(true)
53       setRedirectToLogin(false);
54     }
55     setTimeout(() => {
56       setRedirectToLogin(true);
57     }, 1000);
58   }
59 };
60
61 useEffect(() => {
62   onAuthStateChanged(auth, (user) => {
63     if (user) {
64       setEmail(user.email);
65     } else {
66       setEmail(null);
67     }
68   });
69 }, [auth]);
70
71 return (
72   <IonApp>
73     <IonReactRouter>
74       <IonMenu contentId="main-pages">
75         <IonHeader>
76           <IonToolbar>
77             <IonTitle className="app-title">Snippet Menu</IonTitle>
78             <IonButtons slot="end">
79               <IonMenuToggle autoHide>
80                 <IonButton>
81                   <IonIcon icon={closeOutline} size="large"></IonIcon>
82                 </IonButton>
83               </IonMenuToggle>
84             </IonButtons>
85           </IonToolbar>
86         </IonHeader>
87         <IonContent>
88           <IonList>
89             <IonItem lines="none">
90               <IonLabel className="email">{email || "No User Logged In"}</IonLabel>
91               <IonIcon className="icon" icon={personCircleOutline} slot="end" size="large"></IonIcon>
92             </IonItem>
93             <IonMenuToggle autoHide>
94               <IonItem button routerLink="/LoginPage">
95                 <IonLabel className="menu-button">My Code Snippets</IonLabel>
96               </IonItem>
97               <IonItem button routerLink="/AddSnippets" lines="none">
98                 <IonLabel className="menu-button">Add Snippets</IonLabel>
99               </IonItem>
100               <IonItem>
101                 {email !== null && (
102                   <IonButton className="submit hover" expand="block" onClick={signedOut}>Sign Out</IonButton>
103                 )}
104                 {email === null && (
105                   <IonButton className="submit hover" expand="block" routerLink="/Login">Sign In</IonButton>
106                 )}
107               </IonItem>
108             </IonMenuToggle>
109           </IonList>
110           <IonToast isOpen={isOpen} message="Signed Out"
111             onDidDismiss={() => setIsOpen(false)}
112             duration={5000}></IonToast>
113           <Redirect to="/Login" />
114         </IonContent>
115       </IonReactRouter>
116     </IonApp>
117   )
118 }
```

5 Using Firebase Authentication

We also imported and set up Firebase Authentication to handle user login and registration. First step was to create the basic layout of the registration and login pages, which simply required a couple of `IonInput` for the user to be able fill in the respective fields. As well, a button that we knew would execute the authentication process off the user input. We also included a title, simple navigation with a back button, and links on both pages to transition to either page depending on having an account already or not. For better user feedback, we also wanted to include some sort of way to let the user in the know if register or login account was successful or if there was an issue. Thus, we used the `IonToast` feature to display successful registration and login. As well, various alerts if there is any error that occurs such as not inputting proper credentials or a nonvalid email address. Another example would be when registering, the password and confirmation password do not match, thus an alert would show.

With the layout of the pages in place, we created various functions both on the page itself and in the `firebaseConfig.ts` file to utilize for this process. We thought it would be best to keep as many Firebase related functions within the `firebaseConfig` file itself for clarity of the code. Thus, allowing us to call those functions on different pages when required. Firebase docs provide useful information on how to create, sign in and sign out users, by including various needed functions such as the `createUserWithEmailAndPassword(auth, email, password)`, `signInWithEmailAndPassword(auth, email, password)`, and `signOut(auth)`.

Thus, with these in mind, we created asynchronous functions which take the information in the inputs using `getAuth()` to then get a response from Firebase, whether it creates a user or logs in user. If it succeeds, the function returns a Boolean value of `true`; if an error occurs it returns `false` and error messages. This allows us to then program the toast to activate on `true`, to show the successful creation of an account in Firebase, or the successful login. We also use this Boolean to then redirect to the desired page. If it was successful on creation, then it is redirected to login page. If login page is successful, then redirected into the app itself. Just increasing the security of the app and having separate users access to the app itself. We also confirmed our code was functioning, by using `console.log` to see what errors and user information, as well, being able to view the users on the Firebase platform itself.

While we are happy with the outcome of these pages, we recognize there can still be improvements while further developing this app. Such as including more loading elements to show to the user that the app is still working in the background and potentially saving user profiles on login page for speed access of just entering in the password for an account. Below are some screenshots of the `FirebaseConfig` file, login page, and the before/after of register page.

Laurea University of Applied Sciences
27.10.2025

Firestore Configuration and Required Functions

```
src > firebaseConfig > @userSignOut
1 import { initializeApp } from "firebase/app"
2 import { getAuth, createUserWithEmailAndPassword, signInWithEmailAndPassword, signOut } from "firebase/auth"
3 import { getFirestore } from "firebase/firestore";
4
5 const firebaseConfig = {
6   // TODO: Add your Firebase configuration
7   apiKey: import.meta.env.VITE_apiKey,
8   authDomain: import.meta.env.VITE_authDomain,
9   projectId: import.meta.env.VITE_projectId,
10  storageBucket: import.meta.env.VITE_storageBucket,
11  messagingSenderId: import.meta.env.VITE_messagingSenderId,
12  appId: import.meta.env.VITE_appId
13 };
14
15 const app = initializeApp(firebaseConfig);
16 const auth = getAuth(app);
17 const db = getFirestore(app);
18
19 export async function loginUser(email: string, password: string) {
20
21   try {
22     const userCredential = await signInWithEmailAndPassword(auth, email, password);
23     const user = userCredential.user;
24     console.log("logged in: ", user);
25     return true;
26   } catch (error: any) {
27     const errorCode = error.code;
28     const errorMessage = error.message;
29     console.error(errorCode, errorMessage);
30     alert(errorMessage);
31     return false;
32   }
33 };
34
35 export async function registerUser(email: string, password: string) {
36   try {
37     const userCredential = await createUserWithEmailAndPassword(auth, email, password);
38     const user = userCredential.user;
39     console.log("User Registered: ", user);
40     return true;
41   } catch (error: any) {
42     const errorCode = error.code;
43     const errorMessage = error.message;
44     console.error(errorCode, errorMessage);
45     alert(errorMessage);
46     return false;
47   }
48 };
49
50 export async function userSignOut() {
51   try {
52     await signOut(auth);
53     return true;
54   } catch (error: any) {
55     const errorCode = error.code;
56     const errorMessage = error.message;
57     console.error(errorCode, errorMessage);
58     alert(errorMessage);
59     return false;
60   }
61 }
62
63 export { db };
```

Registration Page (un-updated)

```
pages > src > components > registration
1 import { InputField, InputContent, InputHeader, InputInput, InputLabel, InputPage, InputTitle, InputToast, InputFooter } from "src/components";
2 import React, { useState } from "react";
3
4 const Registration: React.FC = () => {
5   const [email, setEmail] = useState("");
6   const [password, setPassword] = useState("");
7   const [confirmPassword, setConfirmPassword] = useState("");
8   const [isLoading, setIsLoading] = useState(false);
9
10  function registerComplete() {
11    if (password !== confirmPassword) {
12      alert("Passwords DO NOT MATCH");
13    } else {
14      setIsLoading(true);
15      if (email.trim() === "" || password.trim() === "") {
16        alert("Require Email and Password");
17      }
18      console.log(email, password, confirmPassword);
19    }
20  }
21
22  return (
23    <div>
24      <InputPage>
25        <InputHeader>
26          <InputTitle>Registration</InputTitle>
27        </InputHeader>
28        <InputContent>
29          <InputForm>
30            <InputField>
31              <InputLabel>Enter Email</InputLabel>
32              <InputInput type="text" value={email} onChange={e => setEmail(e.detail.value)} />
33            </InputField>
34            <InputField>
35              <InputLabel>Enter Password</InputLabel>
36              <InputInput type="password" value={password} onChange={e => setPassword(e.detail.value)} />
37            </InputField>
38            <InputField>
39              <InputLabel>Confirm Password</InputLabel>
40              <InputInput type="password" value={confirmPassword} onChange={e => setConfirmPassword(e.detail.value)} />
41            </InputField>
42            <InputButton>
43              <InputContent>Register</InputContent>
44              <InputLabel>Register</InputLabel>
45              <InputInput type="button" value="Register" />
46            </InputButton>
47          </InputForm>
48        </InputContent>
49      </InputPage>
50    </div>
51  );
52 }
53
54 export default Registration;
```

Registration Page (Updated)

```

1 // RegistrationPage X
2
3 const Registration = React.FC = () => {
4   const [email, setEmail] = useState('');
5   const [password, setPassword] = useState('');
6   const [confirmPassword, setConfirmPassword] = useState('');
7   const [isLoading, setIsLoading] = useState(false);
8   const [redirectLogin, setRedirectLogin] = useState(false); // for redirect
9
10  // async function registerComplete() {
11    // console.log(email, password, confirmPassword);
12    // console.log(password, confirmPassword);
13
14    // Checking requirements are met
15    if (email.trim() === '' || password.trim() === '' || confirmPassword.trim() === '') {
16      alert("Require Email and Password");
17      return;
18    }
19    if (password !== confirmPassword) {
20      alert("Passwords DO NOT MATCH");
21      return;
22    }
23
24    const resolution = await registerUser(email, password);
25    if (resolution) {
26      setIsLoading(true);
27      setTimeout(() => {
28        setRedirectLogin(true);
29      }, 1000);
30    }
31  }
32
33  if (redirectLogin) {
34    return <Redirect to="/login" />; // Conditional redirecting
35  }
36  return (
37    <IonPage>
38      <IonHeader>
39        <IonToolbar>
40          <IonButtons slot="start">
41            <IonBackButton className="back-button" defaultHref="/" />
42          </IonButtons>
43          <IonTitle className="registration">Register</IonTitle>
44        </IonHeader>
45        <IonContent className="ion-padding">
46          <IonItem className="placeholder">
47            <IonInput placeholder="Enter Email" onChange={e => setEmail(e.detail.value)} />
48          </IonItem>
49          <IonItem className="placeholder">
50            <IonInput type="password" placeholder="Enter Password" onChange={e => setPassword(e.detail.value)} />
51          </IonItem>
52          <IonItem className="placeholder">
53            <IonInput type="password" placeholder="Confirm Password" onChange={e => setConfirmPassword(e.detail.value)} />
54          </IonItem>
55          <p className="register-account">Already have an Account? <a href="/login">Click here</a> to login</p>
56          <IonButton className="submit-button" type="button" onClick={registerComplete}>Submit</IonButton>
57          <IonToast isOpen={isLoading} message="User Created Successfully"
58            onDismiss={() => setIsLoading(false)}
59            duration={1000}>
60          </IonToast>
61        </IonContent>
62      </IonPage>
63    );
64  };
65  export default Registration;

```

Login Page

```

1 // Login X
2
3 const Login = React.FC = () => {
4   const [email, setEmail] = useState('');
5   const [password, setPassword] = useState('');
6   const [isLoading, setIsLoading] = useState(false);
7   const [isLoggedIn, setIsLoggedIn] = useState(false);
8
9   // async function loginComplete() {
10    // console.log(email, password);
11    if (email.trim() === '' || password.trim() === '') {
12      alert("Require Email and Password");
13      return;
14    }
15
16    const resolution = await loginUser(email, password);
17    if (resolution) {
18      setIsLoading(true);
19      setTimeout(() => {
20        setIsLoggedIn(true); // This triggers redirect below
21      }, 1000); // 1 second delay
22    }
23  }
24
25  if (isLoggedIn) {
26    return <Redirect to="/landingPage" />; // Conditional navigation
27  }
28  return (
29    <IonPage>
30      <IonHeader>
31        <IonToolbar>
32          <IonButtons slot="start">
33            <IonBackButton className="back-button" defaultHref="/" />
34          </IonButtons>
35          <IonTitle className="login">Login</IonTitle>
36        </IonHeader>
37        <IonContent className="ion-padding">
38          <IonItem className="placeholder">
39            <IonInput placeholder="Enter Email" onChange={e => setEmail(e.detail.value)} />
40          </IonItem>
41          <IonItem className="placeholder">
42            <IonInput type="password" placeholder="Enter Password" onChange={e => setPassword(e.detail.value)} />
43          </IonItem>
44          <p className="login-account">Account doesn't have an Account? <a href="/registration">Click here</a> to Register</p>
45          <IonButton className="login-button" type="button" onClick={loginComplete}>Login</IonButton>
46          <IonToast isOpen={isLoading} message="Successfully logged in"
47            onDismiss={() => setIsLoading(false)}
48            duration={1000}>
49          </IonToast>
50        </IonContent>
51      </IonPage>
52    );
53  };
54  export default Login;

```

6 Working with Firestore

We also created a database for our project in the Firebase Console. Creating the database was straightforward, as it included 3 simple steps. We chose a simple test database and selected Europe as the cloud region and then imported getFirestore from Firebase and initialized constants in our app that connect our app to Firebase.

When it comes to integrating the database, we exported db from our firebaseConfig.ts file so that it could be used in the Landing and Add Snippet pages. In both pages, we imported collection and getDocs from Firebase/Firestore to be used for snippet creation and fetching. This allowed us to store new snippets in the Firestore collection and get them from the database so that they can be displayed on the Landing page.

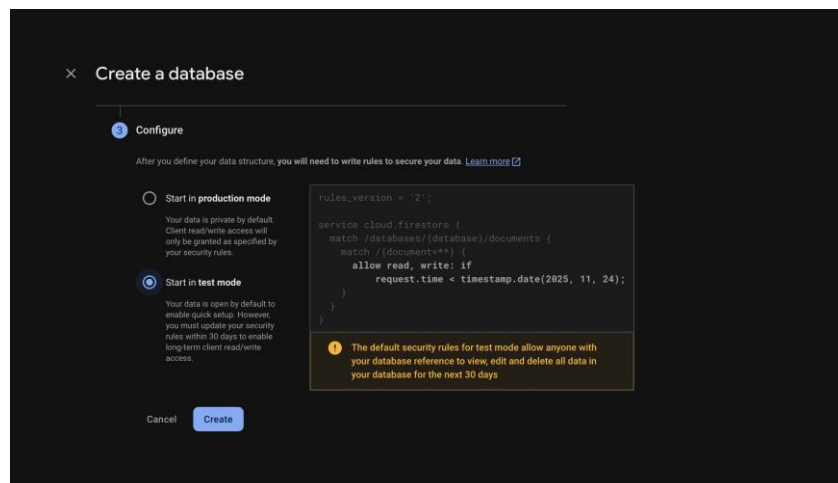
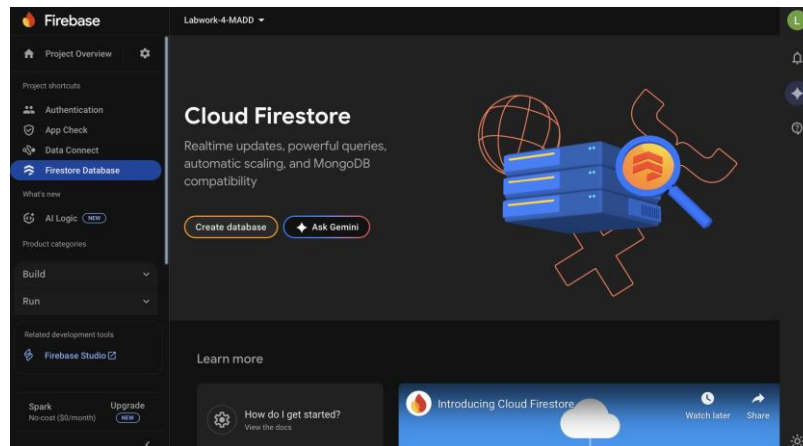
In both of these pages, we imported collections and getDocs from the Firebase/Firestore. Collections are containers that are used for storing data inside the Firestore. It is possible to have multiple collections inside Firestore, however, for this project we only created one. One important note here is that we did not need to add collection manually - it was created automatically when we were testing the app functionality. Another note is that we did not include user specific IDs for the individual snippets, so at the moment the app displays all the snippets from the database. However, this feature is on our to-do list for the future improvements.

We used useState to handle saving snippets and defined a function to manage user input and error handling. The landing page uses useState to update the snippet array, while useEffect is used to fetch data from Firestore and keep the app synchronized with the database, displaying the latest version. Inside useEffect, we defined asynchronous function called fetchSnippets which is getting snippets from the database using getDocs function.

In add Snippets page, we first declared state variables and then defined functions for saving snippets. Inside handleSave function, we set alerts to inform the user if the code was successfully saved but also added some error handling and conditions (for example if the title of code fields is empty, the snippet cannot be saved).

We also used IonFab (Floating Action Button) for wrapping our plus button in the Landing Page. The goal was to have a plus button at the bottom right part of the screen that will float over other content and stayed fixed, so that users can click on it and go straight to Add Snippets page. Using IonFab was much simpler compared to CSS configuration of the similar properties.

Creating Database



Landing page (interface and functions)

```
import {
  IonList,
  IonCard,
  IonCardHeader,
  IonCardTitle,
  IonCardContent,
  IonFab,
  IonButton,
  IonIcon,
} from "@ionic/react";
import { addOutline } from "ionicons/icons";
import { collection, getDocs } from "firebase/firestore";
import { db } from "../firebaseConfig";

interface Snippet {
  id?: string;
  title: string;
  code: string;
  explanation: string;
}

const LandingPage: React.FC = () => {
  const [snippets, updateSnippets] = useState<Snippet[]>([]);

  useEffect(() => {
    const fetchSnippets = async () => {
      const snippetsFromDB = await getDocs(collection(db, "snippets"));

      const snippetsList: Snippet[] = snippetsFromDB.docs.map((doc) => ({
        id: doc.id,
        ...(doc.data() as Snippet),
      }));

      updateSnippets(snippetsList);
    };

    fetchSnippets();
  }, []);
```

Landing page (return section)

```
src > pages > LandingPage.tsx > LandingPage
28 const LandingPage: React.FC = () => {
44   }, []);
45
46   return (
47     <IonPage>
48       <IonHeader>
49         <IonToolbar>
50           <IonTitle>My Code Snippets</IonTitle>
51         </IonToolbar>
52       </IonHeader>
53
54       <IonContent className="ion-padding">
55         {snippets.length === 0 ? (
56           <p>No snippets saved yet. Click the "+" button to add one!</p>
57         ) : (
58           <IonList>
59             {snippets.map((snippet) => (
60               <IonCard key={snippet.id}>
61                 <IonCardHeader>
62                   <IonCardTitle>{snippet.title}</IonCardTitle>
63                 </IonCardHeader>
64                 <IonCardContent>
65                   <pre>{snippet.code}</pre>
66                   <p>{snippet.explanation}</p>
67                 </IonCardContent>
68               </IonCard>
69             ))}
70           </IonList>
71         )}
72
73         <IonFab vertical="bottom" horizontal="end" slot="fixed">
74           <IonButton shape="round" size="large" routerLink="/add-snippets">
75             <IonIcon slot="icon-only" icon={addOutline} />
76           </IonButton>
77         </IonFab>
78       </IonContent>
79     </IonPage>
80   );
```

Add Snippets page

```
App.tsx  LandingPage.tsx U  AddSnippets.tsx U X  Login.tsx M  TS fire
src > pages > AddSnippets.tsx > AddSnippet > handleSave
11  IonItem,
12  IonLabel,
13  } from "@ionic/react";
14  import { collection, addDoc } from "firebase/firestore";
15  import { db } from "../firebaseConfig";
16
17  const AddSnippet: React.FC = () => {
18    const [title, setTitle] = useState("");
19    const [code, setCode] = useState("");
20    const [explanation, setExplanation] = useState("");
21
22    const handleSave = async () => {
23      if (title.trim() === "" || code.trim() === "") {
24        alert("Please fill in title and code!");
25        return;
26      }
27
28      try {
29        await addDoc(collection(db, "snippets"), {
30          title,
31          code,
32          explanation,
33        });
34        alert("Snippet saved!");
35        setTitle("");
36        setCode("");
37        setExplanation("");
38      } catch (error) {
39        console.error("Error saving snippet:", error);
40        alert("Failed to save snippet.");
41      }
42    };
43  };
44
```

```
63  <IonLabel position="stacked">Code</IonLabel>
64  <IonTextarea
65    value={code}
66    placeholder="Paste your code here"
67    onIonChange={(e) => setCode(e.detail.value!)}
68    autoGrow
69  />
70 </IonItem>
71
72 <IonItem>
73   <IonLabel position="stacked">Explanation</IonLabel>
74   <IonTextarea
75     value={explanation}
76     placeholder="Write an explanation (optional)"
77     onIonChange={(e) => setExplanation(e.detail.value!)}
78     autoGrow
79   />
80 </IonItem>
81
82 <IonButton expand="block" onClick={handleSave}>
83   Save Snippet
84 </IonButton>
85 </IonContent>
86 </IonPage>
87 );
88 };
89
90 export default AddSnippet;
91
```

Laurea University of Applied Sciences
27.10.2025

Firestore configuration for Firestore

```
TS firebaseConfig.ts 2, ↓M, M × App.tsx LandingPage.tsx U
src > TS firebaseConfig.ts > ...
1 import { initializeApp } from "firebase/app";
2 import {
3   getAuth,
4   createUserWithEmailAndPassword,
5   signInWithEmailAndPassword,
6 } from "firebase/auth";
7 import { getFirestore } from "firebase/firestore";
8
9 // Your web app's Firebase configuration
10 const firebaseConfig = {
11   apiKey: "AIzaSyAAj_fYPuXNi8uSi_SgXSvpGjnySAj2pNM",
12   authDomain: "labwork-4-madd.firebaseio.com",
13   projectId: "labwork-4-madd",
14   storageBucket: "labwork-4-madd.firebaseio.com",
15   messagingSenderId: "1008070537501",
16   appId: "1:1008070537501:web:8040d04306570139a0f1f4",
17 };
18
19 const app = initializeApp(firebaseConfig);
20 const auth = getAuth(app);
21 const db = getFirestore(app);
22
```

Conditional redirecting after logging in

```
async function loginComplete() {
  if (email.trim() === "" || password.trim() === "") {
    alert("Require Email and Password");
    return;
  }

  const resolution = await loginUser(email, password);

  if (resolution) {
    setIsOpen(true);

    setTimeout(() => {
      setLoggedIn(true); // This triggers <Redirect> below
    }, 1000); // 1 second delay
  }
}

if (loggedIn) {
  return <Redirect to="/LandingPage" />; // Conditional navigation
}
```


7 Styling and UX

Apart from developing the main functionalities, we also focused on improving the app's design and user experience (UX).

Inspired by the principles taught in Udacity's UX Design for Mobile Developers course, we developed a Codatlas Code Snippet mobile application with a strong focus on user-centered design. Drawing heavily from the "Ow Zone" case study, we applied key UX strategies such as intuitive navigation, mobile-first layout, and clear visual hierarchy to ensure a seamless and engaging user experience. Navigating the Codatlas application does not require heavy brainwork. The minimum viable product (MVP) or mobile application allows developers to easily browse, save, and in future hopefully share code snippets. These features make it a practical tool for everyday coding tasks. This project not only reinforced our understanding of mobile UX best practices but also demonstrated how thoughtful design-minded developers can significantly enhance usability and user satisfaction.

Some key UX factors to mention:

- Dark mode applied to our mobile application. We consider developers to prefer this like VS Code default theme being modern dark theme.
- Buttons have much padding for handedness and are easily reached for pressing.
- Buttons are mostly centered considering the natural area of the Ow Zone. This enables one-handed usage.
- Navigation is quick and does not require too many steps to login, access snippets page and the page where snippets are created.
- Firebase integration makes it a practical solution for developers and other users to access their code snippets anywhere.

The following screenshots demonstrate the before CSS and after CSS UX design:



Figure: Home page before CSS styling.

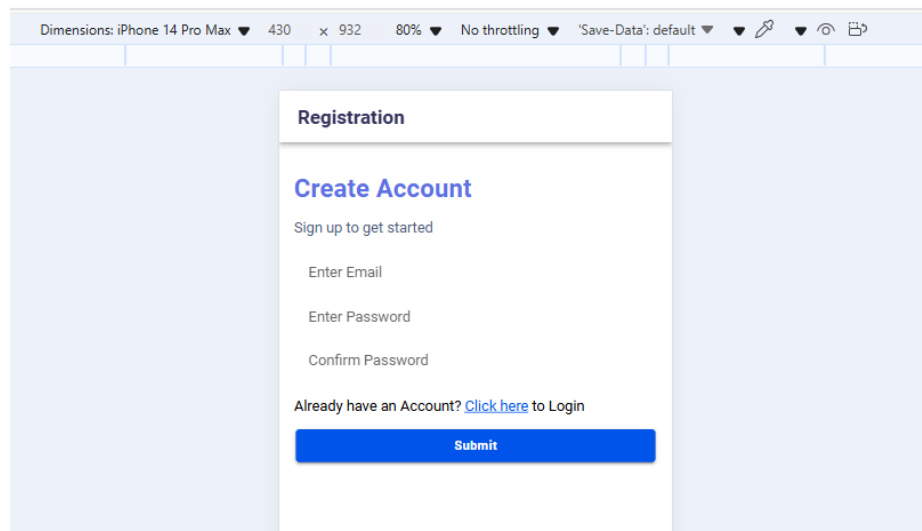


Figure: Register page before CSS styling.

Laurea University of Applied Sciences
27.10.2025

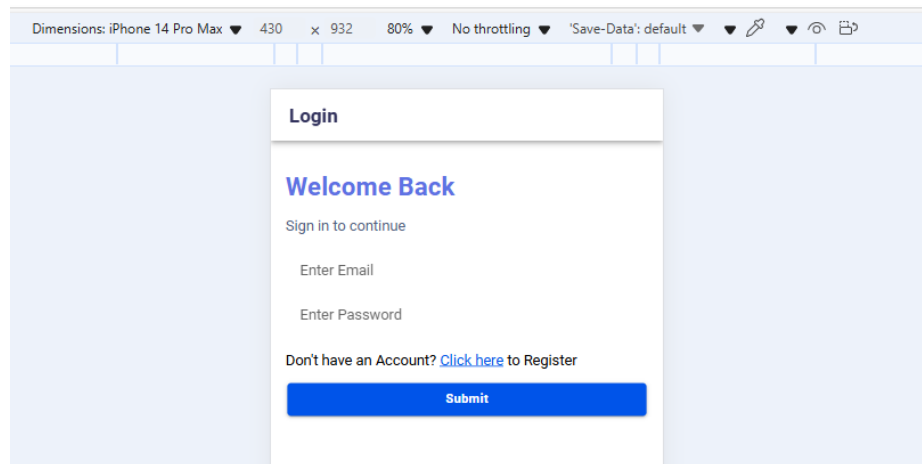


Figure: Login page before CSS styling.

The following screenshots show the output of post CSS styling:

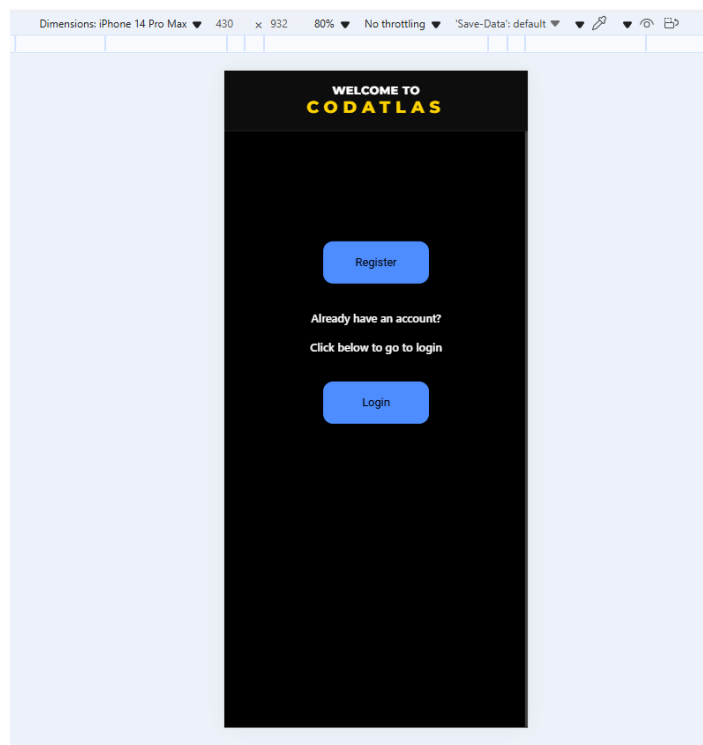


Figure: Home page after CSS styling. Buttons within OW Zone natural zone. Hovering effect applied to buttons which display text in white also for contrast and user identifying where they are navigating. Hover effect is applied to all buttons in other pages.

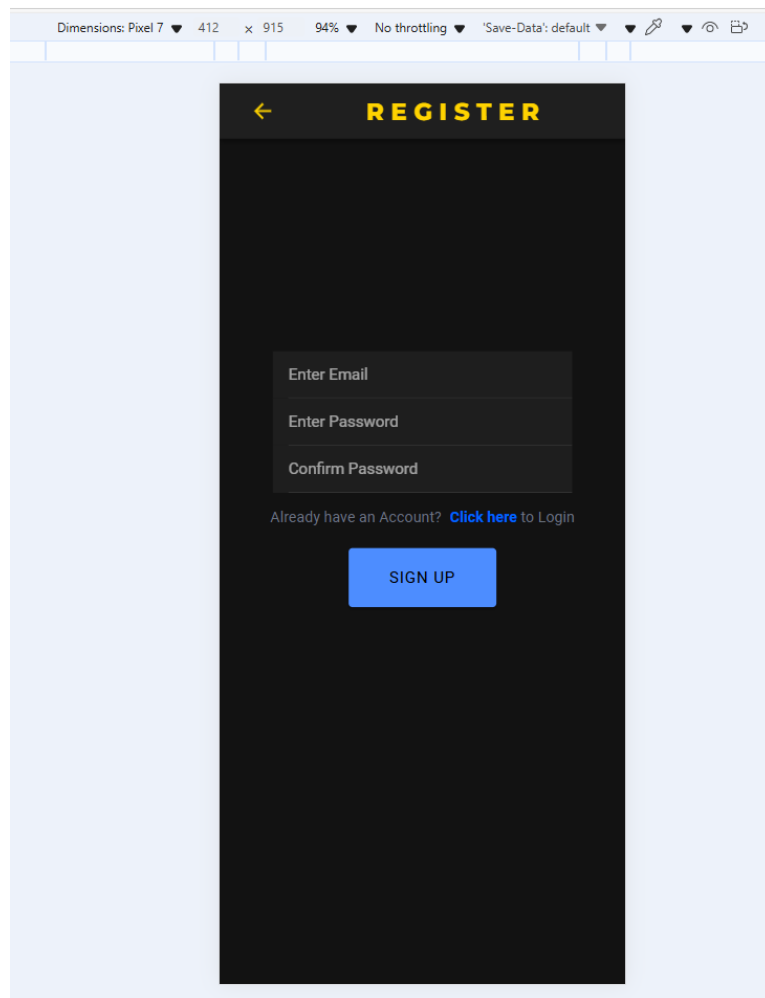


Figure: Register page. Buttons and placeholders in Ow Zone natural zone.

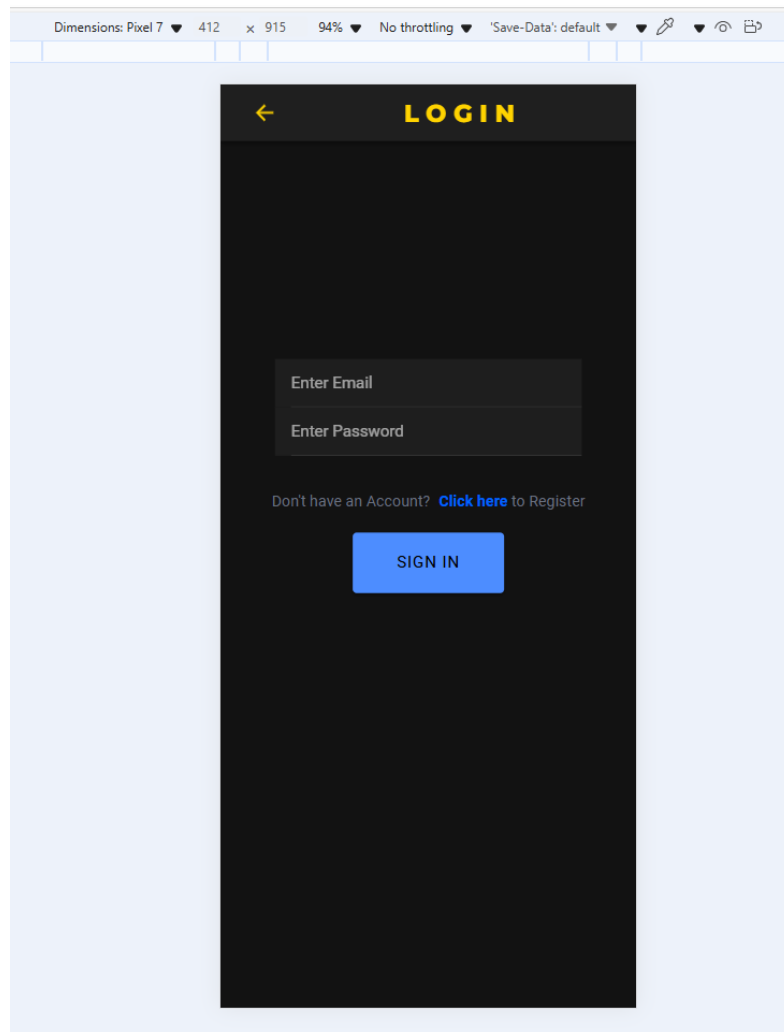


Figure: Login page following the Ow Zone principles of design for mobile applications. The buttons have much padding and placeholders together with buttons are in center within the natural zone of Ow Zone which enables handedness. Users are able to navigate using one hand.



Figure: My Code Snippets page demonstrating clear visual separation of code snippets with headings being colored and snippets being separated with horizontal lines. If overflow, vertical scroll function sets in.

Laurea University of Applied Sciences
27.10.2025

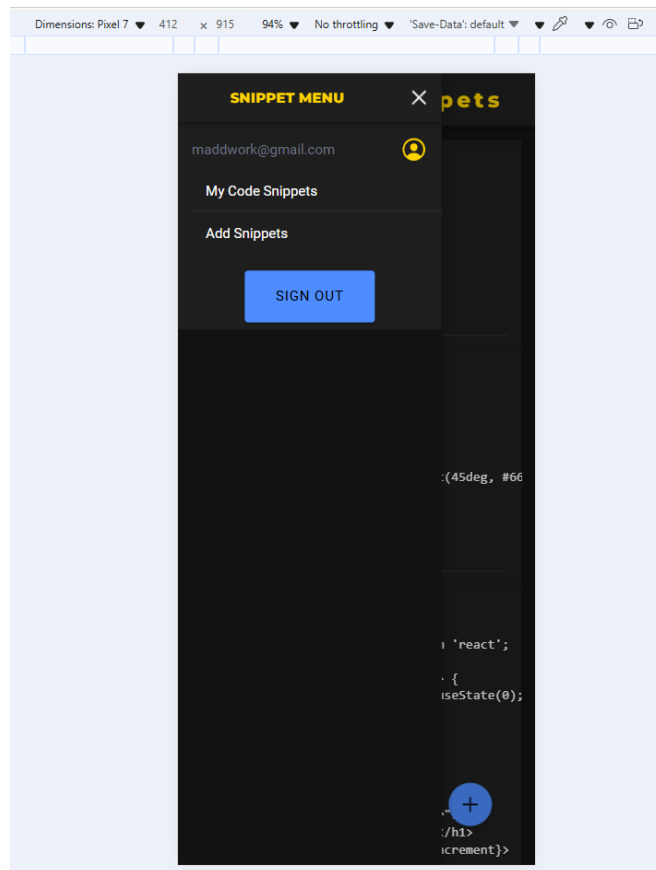


Figure: Snippet Menu navigation provides seamless UX and visual hierarchy of pages. The product usage does not require heavy navigation. Once again, the padding of the “Sign out” button follows the principles of UX design for developers by Udacity which mentions that having much padding helps better with handedness.

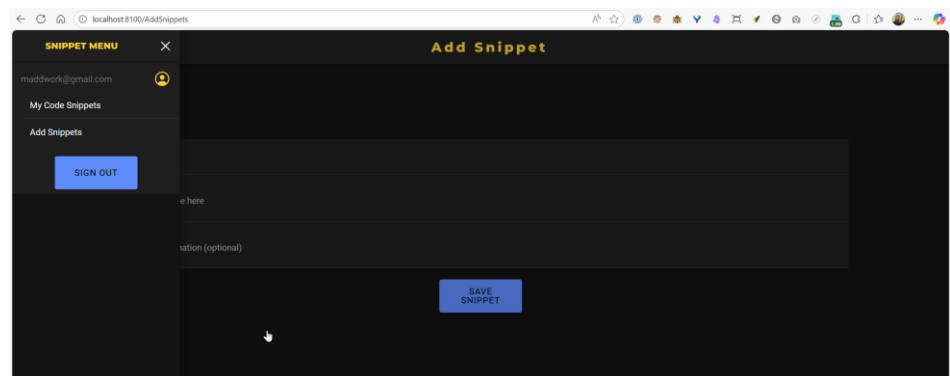


Figure: Snippet menu seen in desktop version.

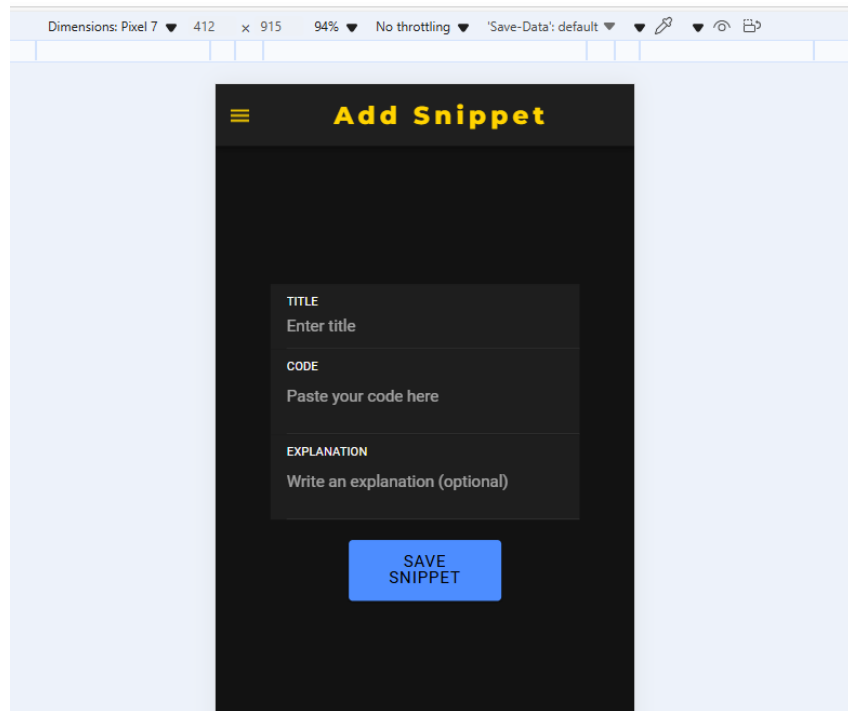


Figure: Add Snippet page which once again follows the Ow Zone principles of design. Buttons have much padding, and placeholders and buttons are aligned in center for handedness.

Laurea University of Applied Sciences
27.10.2025

References

Cloud Firestore Data Model. 2025. Firebase. Accessed 24 October 2025. <https://firebase.google.com/docs/firestore/data-model>

Didin J. 2025. Ionic React Tutorial: Build Firebase Dashboard Mobile App. Djamware. Accessed 24 October 2025. <https://www.djamware.com/post/5e9285ea886d0f77a535f1a2/ionic-react-tutorial-build-firebase-dashboard-mobile-app>

Mehul – Codedamn. 2020. Ionic + React + Firebase #14: Creating Register Page. YouTube. Accessed 27 October 2025. https://www.youtube.com/watch?v=74pAF2Bnfyw&list=PLYxzS__5yYQlhvyLXSKhv4oAvI06MInSE&index=14.

TechAssembler. 2020 Ionic 5 - 18 ion fab. Accessed 24 October 2025. https://youtu.be/7pou_R_J8is?si=mxK-HM6u1jP7L8HB

Udacity's UX Design for Mobile Developers [Mobile UX Design for Developers | Udacity](#)